Applied A.I. Solutions Development Program

Deep Learning II

Waste Classification Service

Group 8

Bikash Giri (101575097)

Chia-Wei Chang (101570243)

Diparshan Bhattarai (1014947379)

Hsi-Teng Liu (101576074)

Gavriel Kirichenko (101119609)

Abdul-Rasaq Omisesan (101571156)

Callum Arul (101585383)

Friba Hussainyar (101591222)

Background and Problem Statement

Recycling requires every person to categorize the waste before disposal. However, the task of sepearting waste into categories is confusing since many products are composite in nature, for that an object could have different parts that belong to different categories. Moreover, the recycling rules are different across different countries and regional governments. The kinds of wastes can be recycled and how they would be collected and processed are all depended on the capabilities of the facilities. Communicating to the population on how to sort wastes is difficult due to the varities of wastes we produce daily which result in many that should be recycled end up in non-recyclable bins and vise versa. The project aims to provide a service which when a picture of wastes is provided will classify the objects in it so a user can correctly categorize the waste in hand.

The Dataset:

For this project, we used the Garbage Classification 3 dataset, which is publicly available on Roboflow Universe. This dataset is specifically designed for object detection tasks related to waste classification, making it highly relevant to our goal of building AI-powered solutions for automated garbage identification.

The dataset contains a total of 10,464 images, each annotated with bounding boxes that label various types of waste materials. These annotations enable object detection models to learn both the location and category of objects within each image. The dataset includes the following six object classes:

- Cardboard
- Glass
- Metal
- Paper
- Plastic
- Trash (general waste)

These classes reflect common types of recyclable and non-recyclable materials, providing a realistic foundation for training deep learning models for environmental applications.

The annotations are available in standard formats such as YOLO, COCO, and Pascal VOC. For our implementation, we used the YOLO annotation format, which is lightweight and well-suited for object detection models like YOLOv8. The images in the dataset were collected from real-world scenarios, featuring diverse backgrounds, lighting conditions, and object placements, which helps improve the robustness and generalizability of the trained models.

The dataset is open-source and publicly licensed for educational and research use, allowing for free experimentation and development. It is ideally suited for training modern deep learning

architectures such as YOLO, Vision Transformers (ViT), and Faster R-CNN, and supports practical applications like smart recycling bins, automated waste sorting systems, and environmental monitoring.

Although the dataset comes with a default train/validation/test split, we performed a custom resplitting process to ensure a balanced distribution of object classes and improve the fairness of model evaluation. This step is detailed in the following section of the report.

YOLOv8 Model Implementation (Gavriel Kirichenko)

Model Overview:

This stage of the project involved the implementation of a **YOLOv8 (You Only Look Once v8)** object detection model to classify waste into six categories: Cardboard, Glass, Metal, Paper, Plastic, and Trash. YOLOv8 was selected due to its efficiency in real-time object detection and strong performance on multi-class datasets. The implementation was carried out using **Google Colab with GPU acceleration (CUDA-enabled)** to facilitate faster training and evaluation. The ultralytics library was used for YOLOv8 training, while **Optuna** was applied for automated hyperparameter optimisation.

Hyperparameter Tuning with Optuna:

Automated hyperparameter optimisation was performed using **Optuna** to maximise detection performance. The following hyperparameters were tuned:

- **Learning rate (lr0)**: Controlled convergence speed.
- **Momentum**: Balanced gradient updates for improved stability.
- **Weight decay**: Regularised the network to prevent overfitting.
- **Data augmentation parameters**: Included Mosaic augmentation, HSV colour space modifications, flipping, and random erasing to increase generalization.

Each trial trained YOLOv8 for **60 epochs** with early stopping (patience of 10 epochs). **mAP@0.5 (Mean Average Precision)** was used as the primary evaluation metric to identify the best-performing configuration.

Training and Results:

The optimized YOLOv8 model achieved:

- **mAP@0.5: 0.5964**, demonstrating strong object detection capabilities across the six waste classes.
- High performance on visually distinct categories such as **Plastic** and **Paper**.
- Slightly lower performance on visually similar materials such as **Metal** and **Glass**, likely due to overlapping textures and reflective properties.

The **YOLOv8s (small) variant** was utilized for efficient training, initially freezing backbone layers to leverage pre-trained COCO weights for transfer learning. This approach accelerated convergence and improved generalization.

The Dataset Resplit Process (Chia-Wei)

The original dataset contains images of six waste material categories: biodegradable, cardboard, glass, metal, paper, and plastic. While this structure was suitable for training. However, we found that the original train/validation/test split was uneven across classes. During exploratory data analysis, we confirmed slight class imbalances (e.g., fewer PLASTIC and METAL samples).

To ensure fair evaluation, we manually resplit the dataset by first loading all labeled samples and then using stratified sampling to allocate approximately 70% to the training set, 15% to the validation set, and 15% to the test set. This method preserved class distribution across splits and improved the robustness of model evaluation.

Vision Transformer ViT Model (Chia-Wei)

Model Description

The core architecture used in this project was the **Vision Transformer (ViT)**, specifically the vit_tiny_patch16_224 variant provided by the timm library. Vision Transformers are known for their strong performance in image classification tasks, as they treat an image as a sequence of patches and apply transformer-style attention mechanisms rather than convolutions. The implementation was done in PyTorch and using the paid version of Google Colab, and training routines were built around the timm, torchvision, and torchmetrics libraries. For the advanced model, we also introduced advanced data augmentation and loss strategies, including **Mixup**, **Cutmix**, and **label smoothing**, along with SoftTargetCrossEntropy as the loss function and CosineAnnealingLR as the learning rate scheduler.

Training & Evaluation

We trained two versions of the ViT model: a **baseline** and an **advanced (augmented)** version. The baseline model was trained for 10 epochs using simple resizing and normalization as the only transformations. It achieved solid validation performance, with an overall accuracy of ~93%. The advanced model was trained for up to 50 epochs, with early stopping set to halt training if no improvement was seen over 5 epochs. This model employed stronger augmentations such as random cropping, color jitter, grayscale conversion, and random erasing. It also incorporated Mixup, Cutmix, and label smoothing, resulting in better generalization. The final accuracy was around 92%.

Hyperparameter Tuning

Hyperparameter tuning played a key role in the model's improvement. We use **SoftTargetCrossEntropy** to handle uncertainty in class boundaries. Dropout rates were also tuned, setting drop_rate=0.2 and attn_drop_rate=0.1 to reduce overfitting. The learning rate schedule was changed to **CosineAnnealingLR**, which allowed smoother convergence. In addition, the use of **Mixup** and **Cutmix** not only augmented the data but also improved model robustness by encouraging the network to generalize across mixed visual features. These changes led to more stable validation loss, quicker convergence, and better class-wise balance in prediction. Ultimately, this comprehensive tuning pipeline transformed a high-performing baseline into a more reliable, production-ready model.

Faster RCNN Resnet (Hsi-Teng Liu)

Model Description

The model is a builtin model fasterrcnn_resnet50_fpn_v2 from pyTorch. It consists of three parts:

1. Backbone with FPN
2. Regional Proposal Network
3. ROI Heads

Counting only convolution layers, there are 66 layers that are trainable.
Because of its generally positive performance on object detection and classification, we decide to use this architecture and pretrained weight as the baseline for training.

Training

The layers of backbone part can be either trainable of not trainable with 5 layers total. To experiment with custom training from the waste dataset we opened up three layers as trainable. Note that the regional proposal network and ROI heads parts are all trainable. When we ran the training, the program could not finish 1 epoch in 24 hours, it progressed really slowly. Then we decided not to train the backbone part keeping the pretrained weights, but the slow progress remained. It turned out with our resources the training can not be done within a week. The count of trained parameters are too many. Below is parameters counts

	3 Trainable backbone layers	0 Trainable backbone layer
Total Parameters	43,281,778	43,281,778
Trainable Parameters	43,056,434	19,773,746
Non-trainable Parameters	225,344	23,508,032

What Can be Done

We should imitate the model and make a much smaller one. Given enough tries, we should be able to find a model that makes satisfactory predictions.

Real-Time Waste Classification Demo (Gavriel Kirichenko)

Demo Overview:

A demonstration was created to showcase a **real-time waste classification pipeline** that integrates both **YOLOv8 (object detection)** and **Vision Transformer (ViT) classification** models. The demo illustrated how these two models complement each other, combining YOLO's ability to detect multiple waste items within a scene with ViT's high-accuracy classification on cropped objects.

Implementation Details:

- **Mode Switching:**
  The demo incorporated a mode variable, enabling real-time toggling between YOLO detection and ViT classification. This provided flexibility for testing each model independently within the same pipeline.
- **YOLOv8 Detection Mode:**
  - Captured frames from a live webcam feed using **OpenCV**.
  - Applied YOLOv8 inference to detect and localize waste items in real time.
  - Annotated frames with bounding boxes, predicted labels, and confidence scores.
  - Displayed "YOLO Detection" overlays for clear identification of the detection mode.
- **ViT Classification Mode:**
  - Processed full frames without object cropping, simulating single-image classification.
  - Converted frames to RGB, applied preprocessing (resizing, normalization), and passed them through the **ViT model**.
  - Displayed the predicted waste category prominently with "ViT Classification" labels.
- **Annotation and Output:**
  Both modes included annotated visualizations overlaid on the live video feed, and the annotated frames were recorded to an output video file for review.
- **Interactive Interface:**
  The demo also integrated a **Gradio web interface**, allowing users to upload images and select either YOLO or ViT mode for classification, returning annotated outputs instantly.

Model Loading and Setup:

- YOLOv8 weights (best_yolo.pt) were loaded using the **Ultralytics library**.

- ViT was initialized via **timm**, loaded with pre-trained weights (best_vit_model.pth) fine-tuned for six waste categories: **Biodegradable, Cardboard, Glass, Metal, Paper, and Plastic**.
- Image preprocessing included resizing to 224×224, normalization, and tensor conversion for ViT input.

Discussion and Reflection (Abdul-rasaq Omisesan)

Misclassification Observed

While both ViT and YOLOv8 performed well overall, we noted several consistent misclassifications during validation and real-time testing:

- Glass misclassified as Metal: due to similar reflective surfaces and overlapping shapes
- Plastic vs. Cardboard: often confused, especially when plastic was opaque or crumpled
- Trash vs. Paper: small or torn paper pieces were sometimes identified as general trash
- Cardboard vs. Background: YOLOv8 struggled with detecting cardboard against brown or cluttered backgrounds

These errors highlight the limitations of visual similarity and dataset ambiguity. They underscore the need for better data augmentation, more diverse samples, and potentially multimodal input in future versions.

Key Learnings from Model Comparison

We evaluated multiple models ViT, Faster R-CNN, and YOLOv8 for waste classification. The baseline Vision Transformer (ViT Tiny) achieved high training accuracy but lacked generalization due to minimal augmentation. After introducing Mixup, Cutmix, grayscale, and random erasing, the macro recall improved from 0.90 to 0.93, and PLASTIC recall increased from 0.73 to 0.84, resolving critical class confusion with cardboard.

YOLOv8, tuned using the Optuna framework, achieved **mAP@50 = 0.598** and **recall = 0.78**, showing consistent performance and deployment potential. Its confusion matrix highlighted areas needing further tuning, such as CARDBOARD vs. background noise. Although Faster R-CNN was explored as well, its computational cost and long training time made it less suitable for our real-time deployment goals.

Engineering Choices That Made a Difference

- **Loss Function**: SoftTargetCrossEntropy improved ViT performance on ambiguous class boundaries.
- **Regularization**: Dropout settings `drop_rate=0.2` and `attn_drop_rate=0.1` reduced overfitting.
- **Scheduler & Training Control**: CosineAnnealingLR improved convergence, and early stopping (patience = 5) prevented wasted computation.

- **YOLOv8 Tuning**: Key hyperparameters (e.g., learning rate, momentum, augmentations) were optimized through automated search with Optuna.

Process-Level Reflection

Dataset Challenges & Strategy

Our original dataset had class imbalance across validation and test splits (e.g., very few PAPER or BIODEGRADABLE samples). To address this, we applied a stratified 70/15/15 split, ensuring balanced representation and fair evaluation for all classes.

Collaboration and Tools

Team members were divided into focused roles: data preparation, model training (ViT, YOLOv8, Faster R-CNN), evaluation, and reporting. Tools such as Google Colab Pro, PyTorch, timm, and torchvision accelerated experimentation, while checkpointing protected against session timeouts and resource constraints.

The real-time demonstration combined YOLOv8's detection with ViT's classification pipeline. The dual-mode toggle system allowed for comparative testing within a single interface, using OpenCV for live video and Gradio for browser-based image testing. This modular design made it easier to validate model predictions in both live and static environments.

Future Plans

1. Smart BIN Integration

- Deploy the classifier with a webcam/mobile input UI
- Show real-time predictions with confidence scores
- Integrate system into physical waste bins for automated sorting

2. Explainability & Trust

- Use Grad-CAM or ViT attention rollout to visually explain predictions
- Enhance user and stakeholder trust with interpretable AI decisions

3. Data Expansion & Diversity

- Collect more real-world images under varied lighting and cluttered backgrounds
- Use synthetic data augmentation and hard negative mining to boost performance

4. Deployment Optimization

- Evaluate lightweight backbones (e.g., MobileViT, EfficientNet, or quantized YOLO)
- Benchmark real-time inference on low-resource hardware (e.g., Raspberry Pi, Android)

Final Thoughts

This project offered a full-cycle experience in applied AI from handling imbalanced data to evaluating models for real-world reliability. Beyond achieving high accuracy, we optimized for fairness, transparency, and readiness for deployment. The result is not just a trained model, but a prototype that aligns with smart city infrastructure and the mission of the Applied AI Solutions Development program.